Отчет по второму практическому заданию по курсу лекций "Численные методы линейной алгебры"

Новикова Вера, 301 группа

Вариант 7

Постановка задачи

В данной работе реализован явный метод Чебышева с оптимальным набором итерационных параметров для приближенного решения системы уравнений

$$x + Ax = f \tag{1}$$

С помощью теоремы Гершгорина был оценен спектр матрицы системы уравнений. Количество итераций было выбрано таким, чтобы среднеквадратичная норма погрешности не превосходила эту же погрешность в прямом методе, а именно в методе отражений Хаусхолдера (в первом задании был также реализован метод ортогонализации Грама-Шмидта, но так как у него погрешность получилась больше, сравнивала только с методом Хаусхолдера). Также был построен график среднеквадратичной нормы погрешности решения как функции номера итерации метода Чебышева.

Метод решения задачи

Оценка спектра матрицы

Спектр матрицы системы оценим с помощью теоремы Гершговина: найдем минимальное λ_{min} и максимальное λ_{max} собственные значения матрицы системы (1) $B = A + I$ с элементами b_{ij}.

По теореме Гершгорина, каждое собственное значение λ ограничено объединением областей:

$$|\lambda - b_{ii}| \leq \sum_{j=1, j\neq i}^{n} |b_{ij}|, \quad i = 1, n$$

Так как матрица B симметрична и положительно определённая, все её собственные значения положительные, т. е. вещественные, следовательно, данные области являются отрезками, тогда оценка минимального и максимального собственного значений производится следующим образом:

$$\lambda_{min} = \min_{1\leq i\leq n} \left(b_{ii} - \sum_{j=1, j\neq i}^{n} |b_{ij}| \right)$$

$$\lambda_{max} = \max_{1\leq i\leq n} \left(b_{ii} + \sum_{j=1, j\neq i}^{n} |b_{ij}| \right)$$

Метод Чебышева

Система (1) с матрицей системы B и правой частью f решается явным методом Чебышёва:

$$\frac{y^k - y^{k-1}}{\tau_k} + By^{k-1} = f, \quad k = 1, ..., m, \quad y_0 = 0$$

где y^k — решение, полученное на k-ой итерации метода (всего m итераций), а τ_k задается соотношением:

$$\tau_k = \frac{\tau_0}{1 - p_0 \cos\left(\frac{\pi}{2m}\theta_m^{(k)}\right)}, \quad k = 1, ..., m$$

$$\tau_0 = \frac{2}{\lambda_{\min} + \lambda_{\max}}$$

$$p_0 = \frac{1 - \chi^{-1}}{1 + \chi^{-1}}$$

где χ — число обусловленности матрицы B: $\chi = \frac{\lambda_{max}}{\lambda_{min}}$, а $\theta_m^{(k)}$ — k-ый элемент множества оптимальных параметров θ_m, для $m = 2^p$, $p \in \mathbb{N}$, которое строится следующим образом: Пусть у нас уже построено множество θ_l:

$$\theta_l = \{\theta_l^1, \theta_l^2, \ldots, \theta_l^l\},$$

тогда θ_{2l} строится следующим образом:

$$\theta_{2l} = \{\theta_{2l}^{(2i)} = 4l - \theta_l^{(i)}, \quad \theta_{2l}^{(2i-1)} = \theta_l^{(i)}, \quad i = 1, \ldots, l, \}, \quad l = 1, 2, 4, \ldots, 2^{p-1}$$

$$\theta_1 = \{1\}$$

Листинг программы

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <stdexcept>
#include <string>
#include <cmath>
#include <chrono>
#include <random>
#include <algorithm>
#include "linearalgebra.h"


void householder(const std::vector<std::vector<double>>& A,
                 std::vector<std::vector<double>>& Q,
                 std::vector<std::vector<double>>& R,
                 int n)
{
    R = std::vector<std::vector<double>>(A);
    double sign;
    double refl_length;
    double refl_val;
    double norm;
```

```
double norm_sqrt;
std::vector<double> scal(n, 0);
std::vector<double> refl_coeff(n, 0);
std::vector<double> w(n,0);

for(int j = 0; j < n - 1; j++) {
    sign = (R[j][j] >= 0.0) ? 1.0 : -1.0;
    norm_sqrt = 0;

    for(int i = j; i < n; i++) {
        norm_sqrt += R[i][j] * R[i][j];
    }

    norm = sqrt(norm_sqrt);
    refl_length = sign * norm;
    refl_val = R[j][j] + refl_length;
    norm_sqrt = norm_sqrt + refl_val * refl_val - R[j][j] * R[j][j];
    norm = sqrt(norm_sqrt);
    w[j] = refl_val / norm;
    refl_coeff[j] = 2.0 * w[j];

    for(int i = j + 1; i < n; i++) {
        w[i] = R[i][j] / norm;
        refl_coeff[i] = 2.0 * w[i];
    }
    sign = (R[j][j] >= 0.0) ? -1.0 : 1.0;

    for(int p = j; p < n; p++){
        scal[p] = 0;
        for(int k = j; k < n; k++) {
```

```
                scal[p] += R[k][p] * w[k];
            }
        }

        for(int p = j; p < n; p++) {
            for(int k = j; k < n; k++) {
                R[p][k] = sign * (R[p][k] - refl_coeff[p] * scal[k]);
            }
        }
    }
    for(int i = 0; i < n; i++) {
        Q[i] = gauss_for_column(i, A, R);
    }
}

class ChebyshevInfo {
    std::vector<double> x;
    std::vector<double> x_sol;
    std::vector<double> f;
    int n;
    double lambda_min;
    double lambda_max;
    double norm;
    double exec_time;
    std::vector<std::vector<double>> A;

public:

    ChebyshevInfo(std::vector<std::vector<double>>& matrix,
                  std::vector<std::vector<double>>& x_)
```

```
{
    n = matrix.size();
    x.resize(n);
    x_sol.resize(n);
    f.resize(n);
    A.resize(n, std::vector<double>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            A[i][j] = matrix[i][j];
        }
    }
    for (int i = 0; i < n; i++) {
        A[i][i]++;
    }
    std::vector<std::vector<double>> f_ = matrix_multiply(A, x_);
    for (int i = 0; i < n; i++) {
        f[i] = f_[i][0];
        x[i] = x_[i][0];
    }
}

void gershgorin()
{
    std::vector<double> min_a(n, 0);
    std::vector<double> max_a(n, 0);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            min_a[i] -= std::abs(A[i][j]);
        }
```

```
        if (A[i][i] >= 0) {
            max_a[i] = -min_a[i];
            min_a[i] += 2 * A[i][i];
        } else {
            max_a[i] = -min_a[i] + 2 * A[i][i];
        }

        min_a[i] = -min_a[i];
    }
    lambda_min = -*std::max_element(min_a.begin(), min_a.end());
    lambda_max = *std::max_element(max_a.begin(), max_a.end());
}

std::vector<int> optimal_indices(int m)
{
    std::vector<int> indices(m, 0);

    if (m == 1) {
        indices[0] = 1;
        return indices;
    } else if (m == 3) {
        indices[0] = 1;
        indices[1] = 3;
        return indices;
    } else {
        std::vector<int> previous_indices = optimal_indices(m / 2);

        for (int i = 0; i < m; i += 2) {
            indices[i] = previous_indices[i / 2];
```

```
            indices[i + 1] = m * 2 - indices[i];
        }

        return indices;
    }
}

void chebyshev(int iters)
{
    auto start = std::chrono::high_resolution_clock::now();
    x_sol = std::vector<double>(n, 0);
    std::vector<double> v(n, 0);
    std::vector<int> ind(n, 0);
    double t_0;
    double cond;
    double p_0;
    double t_n;
    int j = 0;

    gershgorin();
    t_0 = 2.0 / (lambda_max + lambda_min);
    cond = lambda_max / lambda_min;
    p_0 = (1.0 - 1.0 / cond) / (1.0 + 1.0 / cond);
    ind = optimal_indices(iters);

    for (int i = 0; i < iters; i++) {
        j = ind[i];
        t_n = t_0 / (1.0 - p_0 * cos(M_PI / (2.0 * iters) * j));
        v = std::vector<double>(n, 0);
```

```
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                v[i] += A[i][j] * x_sol[j];

        for (int i = 0; i < n; i++)
            x_sol[i] = t_n * (f[i] - v[i]) + x_sol[i];
    }

    std::vector<double> x_err(x_sol);
    for (int i = 0; i < n; i++)
        x_err[i] = x_err[i] - x[i];

    norm = calculate_mean_square_norm(x_err);
    auto stop = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>
    exec_time = duration.count();
}

void print()
{
    std::cout << "Chebyshev" << std::endl;
    std::cout << "Spectrum:" << std::endl;
    std::cout << "lambda_min: " << lambda_min << std::endl;
    std::cout << "lambda_max: " << lambda_max << std::endl;
    std::cout << "Solution error: " << norm << std::endl;
    std::cout << "Relative error: " << norm / calculate_mean_square_norm(
    std::cout << "Time taken: " << exec_time << " milliseconds" << std::e
}

double get_norm()
```

```
    {
        return norm;
    }
};


void write_csv(std::string filename, const std::vector<double> & vals)
{
    std::ofstream myFile(filename);
    for(int i = 0; i < (int)vals.size(); ++i) {
        myFile << vals[i] << "\n";
    }
    myFile.close();
    std::cout << "Created file " << filename << std::endl;
}

int main()
{
    // std::string filename = "test.txt";
    std::string filename = "SLAU_var_7.csv";
    std::ifstream file(filename);

    std::vector<std::vector<double>> A_input;
    std::string line;

    while (std::getline(file, line)) {
        std::stringstream ss(line);
        std::string value;
        std::vector<double> row;
```

```
    while (std::getline(ss, value, ',')) {
        row.push_back(std::stod(value));
    }

    A_input.push_back(row);
}

file.close();

int n = A_input.size();

std::vector<std::vector<double>> Q(n, std::vector<double>(n, 0.0));
std::vector<std::vector<double>> R(n, std::vector<double>(n, 0.0));

// Householder
std::cout << "Householder" << std::endl;
auto start = std::chrono::high_resolution_clock::now();
householder(A_input, Q, R, n);

std::vector<std::vector<double>> x;
std::vector<std::vector<double>> x_sol;
std::vector<std::vector<double>> f;

x = generate_x(n);
f = matrix_multiply(A_input, x);
std::vector<std::vector<double>> Qtf = matrix_multiply(transpose(Q), f);
x_sol = solve_system(R, Qtf, n);

auto stop = std::chrono::high_resolution_clock::now();
std::vector<double> x_err(n);
```

```
    for (int i = 0; i < n; i++) {
        x_err[i] = x[i][0] - x_sol[i][0];
    }
    std::cout << "Solution error: " << calculate_mean_square_norm(x_err) << s
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(sto
    std::cout << "Time taken: " << duration.count() << " milliseconds" << std
    // ------------------------------------------------------------------------
    int iters = 256;
    ChebyshevInfo A_cheb(A_input, x);
    A_cheb.chebyshev(iters);
    std::cout << std::endl;
    A_cheb.print();
    bool GRAPH_INFO = false;
    int N = 9;
    // GRAPH_INFO = true;
    if (GRAPH_INFO) {
        int m = pow(2, N);
        std::vector<double> norms(m, 0);
        for(int i = 1; i <= m; i++) {
            A_cheb.chebyshev(i);
            norms[i - 1] = A_cheb.get_norm();
        }

        write_csv("graphinfo.csv", norms);
    }

    return 0;
}
```

```
Householder
Solution error: 1.40542e-14
Time taken: 13 milliseconds

Chebyshev
Spectrum:
lambda_min: 1
lambda_max: 151.8
Solution error: 1.94786e-15
Relative error: 3.16996e-16
Time taken: 31 milliseconds

Process returned 0 (0x0)   execution time : 0.089 s
Press any key to continue.
```

Результаты

Удалось получить следующие результаты. График построен с помощью python. Для запуска кода используйте скрипт run.sh.

Величина	Значение
Абс. погрешность прямого метода	1.40542×10^{-14}
Абс. погрешность итер. метода	1.94786×10^{-15}
Относительная погрешность итер. метода	3.16996×10^{-16}
Время вычисления (прямой)	13
Время вычисления (итерационный)	31
Оптимальное количество итераций	256
λ_{min}	1
λ_{max}	151.8

Зависимость нормы от количества итераций
Среднеквадратическая норма погрешности решения
Итерация
10^{4}
10^{1}
10^{-2}
10^{-5}
10^{-8}
10^{-11}
10^{-14}
0
100
200
300
400
500